INTRODUCTION TO GENERATIVE AI, THE LLM TOOLING ECOSYSTEM & COMMERCIAL API

Name: _____ **Branch/Year:** _____ **Date:** _____

Warm-Up: What Do You Already Know?

Before we start — let's test your instincts. **No wrong answers, just honest guesses!**

1. What does "Generative AI" actually mean?

A. AI that searches the internet and finds existing answers

B. AI that creates new content (text, images, code, audio)

C. AI that replaces human workers in every industry

D. AI that only works on text, nothing else

2. When a developer wants to build an AI chatbot today, they usually:

A. Train a massive model from scratch on their laptop

B. Use a pretrained model through tools and libraries

C. Copy-paste the entire ChatGPT source code

D. Write the AI in Java only

3. Which of these can Generative AI *currently* create?

A. A short story about a robot gardener

B. A Python function to check even numbers

C. An image of a futuristic classroom

D. All of the above

Part 1: What Is Generative AI?

How a Writer Used AI Without Writing a Single Line of Code

- A writer was stuck on his novel. He used an AI tool to **generate ideas** for characters and plot.
- The AI didn't write the book — it just helped him **get unstuck faster**.
- Today, people use AI tools to create text, images, music, and code — without building AI from scratch.

Key Question: How do developers actually use AI models in their apps?

Definition

Generative AI refers to models that learn patterns from data and generate **new content** — text, images, audio, or code — that resembles the training data.

Key Characteristics:

- Learns **probability distributions** of data
- Produces **new and original** outputs
- Typically uses deep learning architectures (Transformers, Diffusion models)

> **ACTIVITY: Complete the Analogy**
>
> Generative AI is like a **chef** who has _____ thousands of dishes and can now _____ a new dish that feels familiar but has _____ before.
>
> *Word Bank: tasted, cook, never existed*

> **ACTIVITY: Predict the Output!**
>
> If you give this prompt to a Generative AI: `"Write a one-sentence story about a robot learning to garden."`
> Write YOUR predicted output below. Then we'll compare with what the AI actually generates!
>
> **YOUR PREDICTION:**

Part 2: The Four Domains of Generative AI

The same generative principle applies across **different data types**. Let's explore all four:

Domain	What It Creates	How (Tool Used)	Real-World Examples
Text Generation	Stories, emails, summaries	`transformers` library	Chatbots, blog writing
Code Generation	Programs, functions, SQL	`transformers` library	Autocomplete, bug fixing

Domain	What It Creates	How (Tool Used)	Real-World Examples
Image Generation	Art, designs, photos	`diffusers` library	Advertising, game assets
Audio Generation	Speech, music, narration	`transformers` library	Text-to-speech, audiobooks

<u>ACTIVITY:</u> Match the Code to Its Domain

Below are 4 real Python code snippets. For each one:
1. Identify which **domain** it belongs to
2. **Count the lines** of actual code (exclude blank lines)

Snippet A:

```
 from transformers import pipeline

generator = pipeline("text-generation", model="distilgpt2")
output = generator("AI can help students learn by", max_length=40)
print(output[0]['generated_text'])
```

Domain: _____ | Lines of code: _____

Snippet B:

```
 from transformers import pipeline

generator = pipeline("text-generation", model="Salesforce/codegen-350M-mono")
output = generator("def is_even(n):", max_length=60)
print(output[0]['generated_text'])
```

Domain: _____ | Lines of code: _____

Snippet C:

```
 from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5"
)
image = pipe("A futuristic classroom with AI robots").images[0]

image
```

Domain: _____ | Lines of code: _____

Snippet D:

```
 from transformers import pipeline

tts = pipeline("text-to-speech", model="suno/bark-small")
audio_output = tts(
    "Welcome to your first lecture on Generative AI!"
)["audio"]

from IPython.display import Audio
Audio(audio_output, rate=24000)   # Bark-small uses 24 kHz sample rate
```

Domain: _____ | Lines of code: _____

OBSERVATION: Look at your line counts. All four snippets are short (3-4 lines each)! That's the power of the `pipeline()` approach — it handles model loading, tokenization, and output formatting **automatically**.

Quick Check: What is the same generative principle across all four domains?

A. They all use the same Python library

B. Learning patterns from existing data and creating new content

C. They all require GPUs with 80GB memory

D. They all output text

Part 3: Generative AI vs Discriminative AI

Not all AI models are the same. Let's understand the fundamental difference.

OBSERVATION: Think About It: A **Judge** decides if a painting is "good" or "bad." An **Artist** creates a new painting. These are fundamentally different jobs — one classifies, the other creates.

ACTIVITY: Fill In the Comparison Table

Use the word bank to complete the blanks:

Aspect	Discriminative Models	Generative Models
Purpose	_____	_____
Output	_____	_____
Examples	_____	_____
Analogy	_____	_____

Word Bank: Classification, Creation, Label, New content, Logistic Regression / SVM, GPT / Stable Diffusion, Judge, Artist

> **KEY INSIGHT: Discriminative models decide.** Generative models **imagine**.

Quick Check: Your spam filter classifies emails as "Spam" or "Not Spam." Is this Generative or Discriminative?

A. Generative — it generates the label "Spam"

B. Discriminative — it classifies, it doesn't create new emails

C. Both — it does everything

Part 4: Why Models Are Not Trained From Scratch

THE REALITY CHECK: "You Want to Build a Chatbot?"

Let's say your startup wants to build a chatbot. Here's what training from scratch *actually* costs:

Requirement	What You Need	Reality
Training Data	Billions of text documents	Wikipedia alone is only ~4 GB
Compute (GPUs)	Thousands of A100 GPUs	One A100 costs ~$10,000+
Training Time	Weeks to months	GPT-3 took ~34 days on 1000+ GPUs
Estimated Cost	$1 Million — $100 Million	Most startups have ~$10,000 budget

> **OBSERVATION: The Gap:** Training a model from scratch costs **millions**. But using a pretrained model through a library? **Free and takes 3 lines of code.**

ACTIVITY: Fill in the Blanks

Most real-world AI systems rely on _____ models.
This means someone else has already done the _____ training, and you can just _____ the model through tools.

Word Bank: pretrained, expensive, use

Transition Question: If models already exist, where do we get them and how do we use them?

Part 5: The LLM Tooling Ecosystem — Big Picture

Developers don't build AI from scratch — they **assemble solutions** using existing components. Here's the ecosystem:

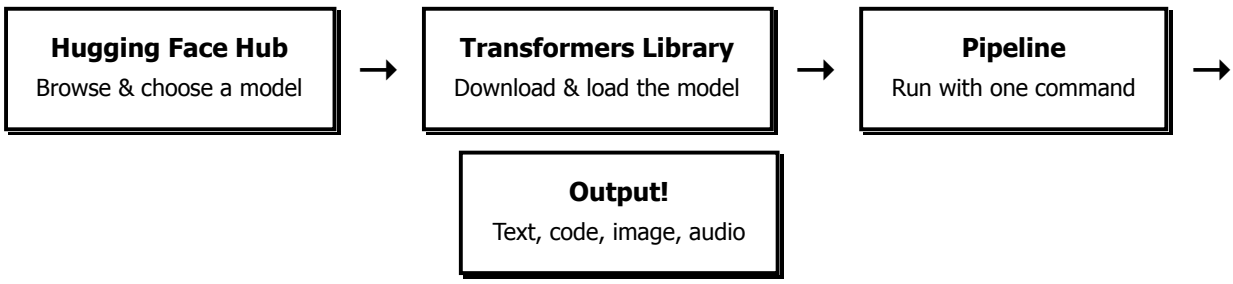
ACTIVITY: Match the Component to Its Analogy

Draw a line connecting each tool to what it's like in real life:

Components	Analogies
1. Hugging Face Hub	**A. Engine that runs the car**
2. Transformers Library	**B. App Store for AI models**
3. Pipeline	**C. Ordering a combo meal**

Your Answers: 1 → _____ 2 → _____ 3 → _____

How They Work Together:



> **KEY INSIGHT:** Developers **assemble** AI solutions using existing components rather than building everything from scratch. Think of it like building with LEGO blocks, not carving from raw stone.

Part 6: Hugging Face & Transformers Library

6.1 Hugging Face Platform

A platform that allows users to:

- **Share** models with the community
- **Discover** models for specific tasks
- **Use** models easily in their applications

6.2 Hugging Face Hub

An online repository containing:

- Pretrained models (500,000+ models!)
- Datasets
- Documentation and model cards

6.3 Transformers Library

A Python library used to:

- **Load** models from the Hub
- **Run inference** locally or on the cloud
- **Apply** models to real-world tasks

Quick Check: What is the relationship between the Hub and the Transformers library?

A. They are the same thing

B. The Hub replaces the Transformers library

C. The Hub stores models; the Transformers library downloads and runs them

D. The Transformers library only works with GPT models

Part 7: Pipeline — The Simplest Interface to LLMs

What Is a Pipeline?

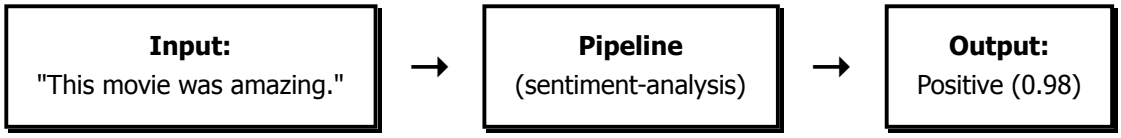
A high-level abstraction that **automatically**:

1. Loads the correct model
2. Loads the correct tokenizer
3. Handles preprocessing
4. Runs inference
5. Returns human-readable output

> **OBSERVATION: Analogy:** Using a pipeline is like ordering a **combo meal** — you say "I want a burger combo" and you get the burger, fries, and drink without worrying about how each was separately prepared.

> **ACTIVITY: Go back to Snippet A (Part 2) and fill in the blanks:**
>
> 1. `"text-generation"` tells the pipeline what _____ to perform.
> 2. `model="distilgpt2"` specifies which _____ to use from the Hub.
> 3. `max_length=40` controls the maximum number of _____ in the output.
> 4. `output[0]['generated_text']` extracts the _____ from the result.
>
> *Word Bank: task, pretrained model, tokens, generated text*

Conceptual Example: Sentiment Analysis



Quick Check: How many lines of actual code does it take to run text generation using a pipeline?

A. 1 line

B. 3 lines (import, create pipeline, generate)

C. 50+ lines

D. You need to write the model architecture first

Part 8: Commercial AI APIs — Why & When

When Your Laptop Isn't Enough

- Your Pipeline code runs **locally** — great for learning and small models.
- But what if you need **GPT-4 level intelligence**? Or your laptop has **no GPU**?
- Commercial APIs let you access the **most powerful AI models in the world** through a simple internet call — no GPU, no setup, no training.

Local Pipeline vs Commercial API

Aspect	Pipeline (Local)	Commercial API (Cloud)
Where it runs	Your machine (CPU/GPU)	Provider's cloud servers
Cost	Free (your electricity)	Pay-per-token
Internet	Only for first download	Always required
Models	Open-source (HF Hub)	SOTA proprietary (GPT, Gemini)
Privacy	Data stays on your machine	Data sent to provider

The Big Three Providers

Provider	Key Model	Strength
OpenAI	GPT-4o, GPT-4o-mini	Industry standard, most widely used
Google AI	Gemini 2.0 Flash	Multimodal (text + images + video)
Groq	LLaMA 3.3 70B	Extreme speed, runs open-source models

WARNING: **API keys are passwords!** Never commit them to GitHub. Never share them publicly. Always store them in **environment variables** or **secret managers**.

Quick Check: Why would you use a Commercial API instead of a local Pipeline?

A. Because local models are always better

B. Because APIs are free to use

C. You need access to state-of-the-art models like GPT-4 that can't run locally

D. APIs don't require any internet connection

Part 9: Using the OpenAI GPT API

Your First API Call

The OpenAI API uses a **chat-based structure** with three roles:

- **system** — Sets the AI's personality/behavior
- **user** — Your question or prompt
- **assistant** — The AI's response (in multi-turn conversations)

Code: Text Generation with GPT

```
 from openai import OpenAI

client = OpenAI(api_key="your-api-key-here")

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful tutor."},
        {"role": "user", "content": "Explain AI in one sentence."}
    ]
)

print(response.choices[0].message.content)
```

ACTIVITY: Understand the Structure

Looking at the code above, fill in the blanks:

1. `OpenAI(api_key=...)` creates a _____ to the API.
2. `model="gpt-4o-mini"` selects a _____ model (fast and cheap).
3. The `"system"` role says: _____.
4. The `"user"` role contains the actual _____.
5. `response.choices[0].message.content` extracts the _____.

Word Bank: connection, smaller, "You are a helpful tutor", question/prompt, generated answer

KEY INSIGHT: OpenAI's API always expects a **messages array** — even for a single question. This is because GPT is designed for **multi-turn conversations**.

Part 10: Using the Google Gemini API

Google's Approach — Simpler for Quick Tasks

Gemini's API is **simpler for basic use** — you can send a plain text prompt without roles.

Code: Text Generation with Gemini

```
import google.generativeai as genai

genai.configure(api_key="your-api-key-here")

model = genai.GenerativeModel("gemini-2.0-flash")
response = model.generate_content("Explain AI in one sentence.")

print(response.text)
```

OBSERVATION: Compare with OpenAI: Gemini's basic call is **simpler** — just pass a string as `contents`. No `messages` array or roles needed for a single question. But for multi-turn chat, Gemini also supports a similar conversation structure.

ACTIVITY: Spot the Differences

Compare the OpenAI (Part 9) and Gemini code. Fill in the table:

Aspect	OpenAI GPT	Google Gemini
Library import	`from openai import OpenAI`	`import google.generativeai as genai`
Input format	_____	_____
Output extraction	_____	_____

Part 11: Using the Groq API (Lightning-Fast Inference)

What Makes Groq Different?

Groq runs **open-source models** (like Meta's LLaMA) on special hardware called **LPUs** — making them incredibly fast. Best part? It uses the **same code format as OpenAI**.

Code: Text Generation with Groq

```
 !pip install groq

from groq import Groq

client = Groq(api_key="your-api-key-here")

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "user", "content": "Explain AI in one sentence."}
    ]
)

print(response.choices[0].message.content)
```

OBSERVATION: Did you notice? Groq's code is **almost identical** to OpenAI's! The only differences are: (1) the import name, and (2) the model name. This is intentional — Groq follows the **OpenAI-compatible format**, making it easy to switch between providers.

Quick Check: What is Groq's key advantage over OpenAI?

A. It has better models than GPT-4

B. Extreme inference speed using specialized LPU hardware

C. It runs models on your local machine

D. It doesn't require an API key

Part 12: Tokens, Cost & Rate Limits

What Are Tokens?

Tokens are the **currency of LLMs**. Every API call costs tokens. A token is roughly **¾ of a word**.

- "Hello" = 1 token
- "Artificial Intelligence" = 2 tokens
- "Explain quantum physics in simple terms" = ~7 tokens

> **ACTIVITY: Estimate the Tokens**
>
> How many tokens (approximately) for each sentence below?
>
> 1. "What is AI?" → _____ tokens
> 2. "Write a Python function to sort a list" → _____ tokens
> 3. A 500-word essay → ~ _____ tokens
>
> *Hint: Multiply word count by 1.3 for a rough estimate*

Pricing Comparison

Provider / Model	Input Cost (per 1M tokens)	Output Cost (per 1M tokens)	Speed
GPT-4o-mini	$0.15	$0.60	Fast
Gemini 2.0 Flash	Free tier available	Free tier available	Fast
Groq (LLaMA 3.3 70B)	$0.59	$0.79	Very Fast

Rate Limits — Speed & Volume Controls

- **RPM** (Requests Per Minute) — How many calls you can make per minute
- **TPM** (Tokens Per Minute) — How many tokens you can use per minute

> **KEY INSIGHT: Cost control tips:** (1) Use cheaper models for simple tasks (`gpt-4o-mini` instead of `gpt-4o`). (2) Set `max_tokens` to limit response length. (3) Cache frequent responses to avoid repeat calls.

Part 13: Pipeline vs Commercial APIs — When to Use What

The Million-Dollar Question

You now know **two ways** to use AI models: (1) Locally via `pipeline()` , and (2) via **Commercial APIs**. When should you use which?

> **ACTIVITY: Choose Your Approach**
>
> For each scenario, write **P** (Pipeline/Local) or **C** (Commercial API):
>
> 1. You're a student learning how text generation works → _____
> 2. You're building a production chatbot for 10,000 users → _____
> 3. You need GPT-4 level quality for a client project → _____
> 4. You want to run AI offline on your laptop → _____
> 5. Your project handles sensitive medical data → _____
> 6. You need to generate 1000 responses per minute → _____

> **KEY INSIGHT: The Rule of Thumb:**
> Use **Pipeline** for learning, experiments, and when privacy matters.
> Use **Commercial APIs** when you need SOTA quality, scale, or can't run models locally.

Quick Check: A startup wants to build a customer support chatbot using GPT-4. They have no GPUs. What should they use?

A. Pipeline — it's simpler and free

B. Commercial API (OpenAI) — they need GPT-4 which can't run locally, and APIs handle scaling

C. Train their own model from scratch

Part 14: Key Terminology

ACTIVITY: Match the Term to Its Definition

Draw a line or write the letter of the correct definition next to each term:

#	Term	Match	Letter	Definition
1	**Training Data**	_____	A	A single piece of text (word or sub-word) that AI processes
2	**Parameters**	_____	B	Text input you give to an AI model
3	**Prompt**	_____	C	The process of running a trained model to get output
4	**Token**	_____	D	The data used to teach a model patterns
5	**Inference**	_____	E	The learned weights inside a model (billions of them!)
6	**LLM**	_____	F	Adapting a pretrained model on a specific, smaller dataset
7	**Fine-tuning**	_____	G	A very large model trained on massive text data (e.g., GPT, LLaMA)
8	**Diffusion Model**	_____	H	A model that generates images by removing noise step by step
9	**API Key**	_____	I	A limit on how many requests or tokens you can use per minute
10	**Endpoint**	_____	J	A secret password used to authenticate with a Commercial API
11	**Rate Limit**	_____	K	A specific URL/function in an API for a particular task (e.g., chat, embeddings)
12	**SOTA**	_____	L	State-of-the-Art — the best-performing model currently available

ACTIVITY: Scenario Challenge: "Generating a Song from a Text Prompt"

Imagine you type: `"Write a jazz song about rainy days"` into an AI music tool.

Which terms from the list above are **directly involved** in this process? Circle or list them below:

YOUR ANSWER (LIST THE TERM NUMBERS):

Part 15: Responsible Use of Generative AI

WARNING: Generative AI is powerful but not perfect. It can produce biased, incorrect, or misleading content. Using it responsibly is not optional — it's essential.

Potential Risks

Risk	What Happens	Example
Biased Outputs	Model reflects biases in training data	AI job screener favors certain demographics
Hallucinations	Model generates confident but false information	"The Eiffel Tower was built in 1723" (Wrong!)
Overconfidence	Users trust AI output without verifying	Student submits AI essay without fact-checking

ETHICAL DILEMMA: Scenario 1: A medical startup uses Generative AI to write patient reports. The AI generates a report that says "Patient shows no signs of concern" — but the patient actually has early symptoms the AI missed.

Question: Who is responsible? The AI? The doctor? The startup?
Your Answer: _____

> **ETHICAL DILEMMA: Scenario 2:** A student uses an AI tool to write their entire assignment. They get an A. Their classmate who wrote it themselves gets a B.
>
> **Question:** Is this fair? Should AI-generated content require a label?
> Your Answer: _____

Responsible Practices

- **Verify outputs** — Always fact-check AI-generated content
- **Use trusted models** — Choose well-documented, community-reviewed models
- **Avoid blind automation** — Keep humans in the loop for critical decisions

Final Challenge: Bring It All Together

THE CHALLENGE: "Your First AI Product"

Your team has been asked to build a **customer support chatbot** for an e-commerce company. Using everything you learned today, answer the following:

1. Is this a **Generative** or **Discriminative** AI task?
Answer: _____ Why? _____

2. Would you train a model from scratch or use a pretrained model?
Answer: _____

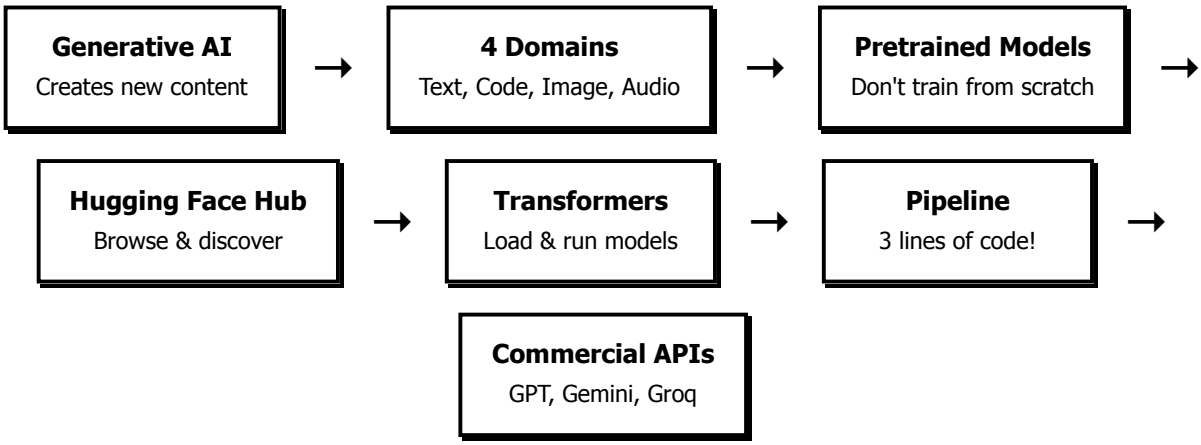
3. Would you use a **Pipeline** or a **Commercial API** for this chatbot? Why?
Answer: _____

4. Which Commercial API provider and model would you choose? (e.g., GPT-4o-mini, Gemini Flash, Groq)
Answer: _____

5. How would you protect your API key in production?
Answer: _____

6. Name one ethical risk you'd need to watch out for:
Answer: _____

Recap: Your Learning Map



KEY INSIGHT: Closing Thought:

Generative AI does not replace human creativity — it **amplifies** it.